Expert Systems With Applications 292 (2025) 128596

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Retrieval-augmented code completion for local projects using large language models

Marko Hostnik [a,b,*], Marko Robnik-Šikonja [a]

[a] Faculty of Computer and Information Science, University of Ljubljana, Večna pot 113, Ljubljana, 1000, Slovenia
[b] Faculty of Mathematics and Physics, University of Ljubljana, Jadranska ulica 19, Ljubljana, 1000, Slovenia

ARTICLE INFO

2020 MSC:
68T50
68T07

Keywords:
Large language models
Code completion
Retrieval-augmented generation
In-context retrieval

ABSTRACT

The use of large language models (LLMs) is becoming increasingly widespread among software developers. However, privacy and computational requirements are problematic with commercial solutions and the use of LLMs. In this work, we focus on using relatively small and efficient LLMs with 160M parameters that are suitable for local execution and augmentation with retrieval from local projects. We train two open transformer-based models, the generative GPT-2 and the retrieval-adapted RETRO, on open-source Python files, and empirically compare them, confirming the benefits of embedding-based retrieval. Furthermore, we improve our models' performance with In-context retrieval-augmented generation (RAG), which retrieves code snippets using the Jaccard similarity of tokens. We evaluate In-context RAG on larger models and determine that, despite its simplicity, the approach is more suitable than using the RETRO architecture. Experimental results indicate that In-context RAG improves the code completion baseline by over 26 %, while RETRO improves over the similarly sized GPT-2 baseline by 12 %. We highlight the key role of proper tokenization in achieving the full potential of LLMs in code completion.

1. Introduction

Programming with the help of large language models (LLMs) and other artificial intelligence (AI) tools is becoming increasingly widespread (Liang, Yang, & Myers, 2024). Programmers use AI tools (e.g., GitHub Copilot[1]) to suggest next lines while writing code (Guo et al., 2024; Li et al., 2023; Rozière et al., 2024), to write tests and shorter or longer sections of code (Austin et al., 2021; Lu et al., 2021), and to facilitate learning new technologies (Kasneci et al., 2023).

As AI tools prove increasingly useful, code editor developers are rapidly incorporating AI into their products. However, the widespread adoption of AI tools raises concerns about code privacy, as code is transmitted to remote servers for processing by commercial models that can contain hundreds of billions of parameters (Brown et al., 2020). These large models require computational resources far beyond what ordinary consumer hardware provides, which is why a number of solutions that allow LLMs to run efficiently on consumer hardware are being developed.[2]

However, the choice of a model size represents an important trade-off, as the model size has a significant impact on the quality of suggestions, the inference speed, and the utilization of compute and memory resources. While existing approaches have focused on improving suggestions using larger models augmented with retrieval from large retrieval datasets (Borgeaud et al., 2022), our approach differs in using small models augmented with only locally available project files.

By focusing on a single programming language or solving just one specific programming task, such as line completion, smaller models can achieve useful results. Additionally, smaller models can speed up the performance of larger models through speculative decoding, where a larger model is used to correct the errors of a smaller model (Leviathan, Kalman, & Matias, 2023). In our work, we focus on the use of LLMs with fewer parameters to complete lines of Python code. This allows the model to be executed within the code editor on the user's hardware, even without a graphics processing unit (GPU) to speed up the execution.

LLMs take a string as input and predict its most probable continuation. Usually, the input string or *context* is the content of the current file being edited. Programming projects are often composed of many files that refer to each other, so we want to enrich the context with information from other files in the project. However, as the context size of models is limited from a few dozen to a hundred lines of code, we need to find informative data to include in the context. In our work, we augment the context of LLMs with relevant code snippets from other files

* Corresponding author.
E-mail addresses: mh9533@student.uni-lj.si (M. Hostnik), marko.robnik@fri.uni-lj.si (M. Robnik-Šikonja).

[1] https://github.com/features/copilot

[2] An example is available at https://github.com/ggerganov/llama.cpp.

https://doi.org/10.1016/j.eswa.2025.128596
Received 8 August 2024; Received in revised form 1 April 2025; Accepted 11 June 2025
Available online 14 June 2025

in the project, which we search for based on the similarity of vector embeddings, or alternatively based on the Jaccard similarity of tokens. The goal of our work is to experimentally verify the impact of retrieval-augmented LLMs on the success of completing lines of code. We focus on smaller models suitable for inference without a GPU and on retrieval only from local projects.

Our proposed approach has widespread use cases in improving the coding experience for developers. Small coding LLMs can be used to offer intelligent code completion suggestions in various environments, such as in local text editors. In our work, we demonstrate how small LLMs can be improved and made more viable with retrieval-augmented generation (RAG) using a simple Jaccard similarity-based approach that doesn't require expensive indexing for retrieving relevant code snippets from developers' projects. Better code suggestions with smaller LLMs can not only help developers write code faster, but our approach does so while maintaining a focus on privacy without incurring additional costs of commercial LLMs.

Our main contributions are as follows:

- We train and compare 160 million parameter RETRO (Borgeaud et al., 2022) and GPT-2 (Radford et al., 2019) models using two different tokenizers and verify the importance of token healing for code completion.
- We compare In-context RAG with Jaccard-similarity-based retrieval to RETRO with embedding-based retrieval and to baseline models of different sizes.
- We examine the impact of copying from code snippets and the impact of the retrieved snippets' similarities to the retrieval query.
- We experimentally confirm the viability of augmenting small LLMs with retrieval from local projects for local use in code editors.

This paper is organized into six sections. In Section 2, we present related work in retrieval-augmented code completion. In Section 3, we present the main methods used, including an overview of the RETRO architecture (Borgeaud et al., 2022). In Section 4, we describe the setup of our experiments, including the dataset, model training details, and metrics used. In Section 5, we present and analyze the results of our trained models evaluated with retrieval from local projects. We conclude in Section 6, where we also present ideas for further work.

2. Related work

The field of LLMs based on the transformer architecture (Vaswani et al., 2017) has been rapidly developing in recent years, not only for natural language processing but also for the processing of programming code. We present an overview of related work, split into four subsections: LLMs for code, retrieval-augmented LLMs, retrieval-augmented code completion, and project based evaluation of code completion.

2.1. Large language models for code

Code generation is important for tasks such as code completion (Guo et al., 2024; Li et al., 2023; Rozière et al., 2024), generating code from natural language instructions (Austin et al., 2021), translating code from one programming language to another (Lu et al., 2021), and code searching based on natural language or code-based queries (Wang et al., 2023b). Neural models for code completion are mostly based on the decoder part of the transformer architecture and differ mainly in the number of parameters, the size of the context and the training data used (e.g., the proportion of each programming language and inclusion of natural language) (Guo et al., 2024; Li et al., 2023). When training LLMs with longer context sizes, concatenating project files into the input context is becoming increasingly common (Guo et al., 2024; Rozière et al., 2024). This better prepares the model for including related files into the context during model execution.

2.2. Retrieval-augmented language models

The use of retrieval in LLMs is often motivated by the desire to improve models' capabilities in open-domain question answering tasks (Patwardhan, Marrone, & Sansone, 2023). DPR (Karpukhin et al., 2020) and REALM (Guu, Lee, Tung, Pasupat, & Chang, 2020) extract the answer from the retrieved data using a BERT encoder (Devlin, Chang, Lee, & Toutanova, 2019). DPR fine-tunes the weights of the query encoder to better match its outputs with the frozen document encoder embeddings. REALM updates the weights of the knowledge retriever encoder during training, which causes computationally demanding re-indexing of data during training. RAG (Lewis et al., 2020) and FiD (Izacard & Grave, 2021) differ in the processing of the retrieved snippets with additionally fine-tuned encoder-decoder models. Ram et al. (2023) show that retrieval is beneficial even without additional training of models; it is sufficient to append the found snippets to the context of the decoder. kNN-LM (Khandelwal, Levy, Jurafsky, Zettlemoyer, & Lewis, 2020) computes a weighted sum of the predictions from an LLM and a k-nearest neighbors classifier when predicting each token. RETRO (Borgeaud et al., 2022), described in Section 3.1, performs chunk-based retrieval and includes the found chunks in the decoder via cross-attention, which, in combination with a frozen document encoder, enables retrieval throughout the entire training of the model.

2.3. Retrieval-augmented code completion

Approaches for retrieval-augmented code completion are similar to those for natural language based open-domain question answering, with differences in the use of retrieval adapted to code and projects as a whole. ReACC (Lu et al., 2022) (and similarly REDCODER (Parvez, Ahmad, Chakraborty, Ray, & Chang, 2021)) combines retrieval with a code-adapted encoder, GraphCodeBERT (Guo et al., 2021), and classic BM-25 search (Harman, 1995), incorporating found snippets into the context. Shrivastava, Larochelle, and Tarlow (2023) use a trained classifier to select the most relevant code snippet from the project to include into the context. RepoCoder (Zhang et al., 2023b) demonstrates retrieval based on the Jaccard similarity of tokens for retrieving line-based snippets from the project and utilizes the generated code to improve subsequent retrieval. This approach is the motivation for our approach described in Section 3.3. RepoFormer (Wu, Ahmad, Zhang, Ramanathan, & Ma, 2024) enhances RepoCoder with adaptive retrieval triggered by the model with a special token. DRAG (Shapkin et al., 2024) extends the model's vocabulary with new tokens from entities in the project that the model can use while generating code.

2.4. Project based evaluation of code completion

There are several datasets for evaluating code completion. HumanEval (Chen et al., 2021) involves completing functions from given function names and docstrings; CodeXGLUE (Lu et al., 2021) contains many datasets for various tasks, including line completion. These datasets only evaluate general code completion abilities within a single file, whereas real projects contain multiple interrelated files. Consequently, datasets for project-level completion have begun to emerge, often as an addition for evaluating the developed approach (Shapkin et al., 2024). RepoBench (Liu, Xu, & McAuley, 2024) and CrossCodeEval (Ding et al., 2023) compile evaluation sets by pre-selecting useful code snippets from related files that the model should include in the context. On the other hand, RepoEval (Zhang et al., 2023b) – more in Section 4.3 – contains entire projects in addition to the marked lines that need to be completed, which allows the use of custom implementations of retrieval from the project and consequently any model.

3. Code line completion methods

In this section, we describe our methodology for code line completion based on LLMs. We first describe the retrieval-enhanced RETRO approach, followed by our improvements, the first concerning the tokenization, and the second using context-based retrieval-augmented generation.

3.1. The RETRO approach

RETRO (Retrieval-Enhanced Transformer) is a transformer based autoregressive language model enhanced with additional snippets retrieved from a database (Borgeaud et al., 2022). The approach splits the input token sequence into contiguous fixed-size chunks and finds snippets similar to the previous chunk when processing the current chunk. Introducing retrieval into the model allows adapting the model to new data without updating its weights, which is particularly useful for predicting facts not present in the training set (e.g., names in a local programming project). For our experiments, we make minor modifications to the original RETRO architecture. We change the chunk embedding model from BERT (Devlin et al., 2019) to CodeT5+ (Wang et al., 2023b) and modify the hyperparameters described in Section 4.5. During inference, we modify the input context by adding some padding tokens, using token healing (see Section 3.2) and In-context RAG (see Section 3.3). Our RETRO approach is illustrated in Fig. 1. Below, we explain the details of the approach, relevant for our application.

Language modeling with retrieval. We denote the token vocabulary by the set $\mathbb{V}$ obtained by a text tokenizer (see Section 4.1). The input token sequence $x = (x_1, \ldots, x_n) \in \mathbb{V}^n$ of length $n \in \mathbb{N}$ is split into $l \in \mathbb{N}$ contiguous chunks $C_1 = (x_1, \ldots, x_m), \ldots, C_l = (x_{n-m+1}, \ldots, x_n)$ of length $m = 64$. The probability of the next token x_i depends only on previous tokens and snippets retrieved by previous chunks:

$$P\left(x_i \mid \theta, (x_j)_{j<i}, \left(\text{RET}_D(C_v)\right)_{v<u_i}\right),$$

where $u_i = \lceil \frac{i}{m} \rceil$ is the index of the chunk containing x_i, $\left(\text{RET}_D(C_v)\right)_{v<1} = \emptyset$, θ are the weights of the model, and $\text{RET}_D(C_u)$ denotes the k retrieved snippets for C_u from the retrieval database D.

Retrieval. The retrieval database is built by splitting the set of input documents D into fixed-size chunks of $m = 64$ tokens. The database stores key-value pairs, where the key is the embedding of a chunk of tokens N, and the value is a pair $[N, F]$, which contains both the chunk N and the next immediate chunk F in the original document. The retriever $\text{RET}_D(C)$ finds the k most similar snippets by comparing the L_2 distance between the embedding of the chunk $\mathcal{M}(C)$ and the database key $\mathcal{M}(N)$.

Borgeaud et al. (2022) use the BERT model (Devlin et al., 2019) for $\mathcal{M}$, while we use the CodeT5+ encoder (Wang et al., 2023b), as explained in Section 4.4. RETRO thus receives as input the input tokens x and the k nearest snippets of each input chunk $\text{RET}_D(C_u) = ([N^1, F^1], \ldots, [N^k, F^k])$. When training the model, we ensure that $\text{RET}_D(C_u)$ does not contain C_{u+1} by filtering out chunks $[N, F]$ that originate from the same training document as x.

The RETRO architecture. The RETRO architecture is based on the encoder-decoder transformer architecture (Vaswani et al., 2017). It differs from the decoder-only GPT-2 architecture by the addition of a bidirectional encoder and a chunked cross-attention mechanism, which operates on chunks and allows the inclusion of retrieved snippets into the decoder. Splitting the input context into fixed-length chunks enables efficient chunk-based retrieval, providing more relevant examples for the entire context and allowing for the encoder to independently process all k retrieved snippets of chunks. Since the retriever itself is not updated during training, it can be updated after the training is complete without modifying the model weights. The RETRO model introduces RETRO layers, which replace certain regular decoder layers. Following Borgeaud et al. (2022), we place RETRO layers at the sixth layer and then at every subsequent third layer (e.g., at layers $6, 9, 12$, if there are 12 layers in total).

Sampling with RETRO. Just as the architecture of RETRO influences the construction of the retrieval database and chunked cross-attention, it also impacts sampling from the model. When sampling from RETRO, we pad the context to make sure it is a multiple of the chunk size m (Wang et al., 2023a). This allows RETRO to retrieve based on the last m tokens in the context, as otherwise predicting within the first chunk would not be improved by retrieval and tokens in the last chunk of the context would not be used for retrieval despite being most relevant to predicting the next token. When generating longer texts, an additional question is how often to perform retrieval. In our experiments, we limit the task to code completion until the end of a line, so retrieval is performed only before the first step of text generation or whenever the context length reaches a multiple of the chunk size.

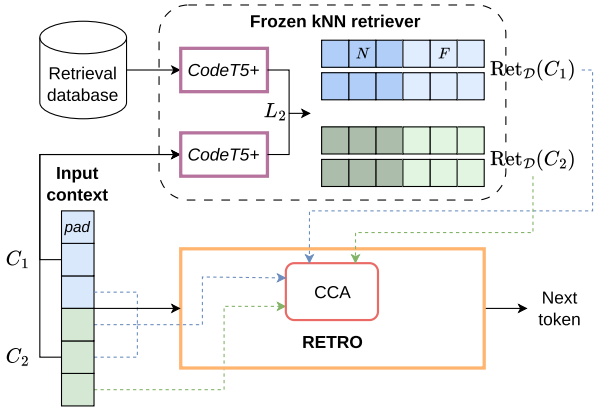
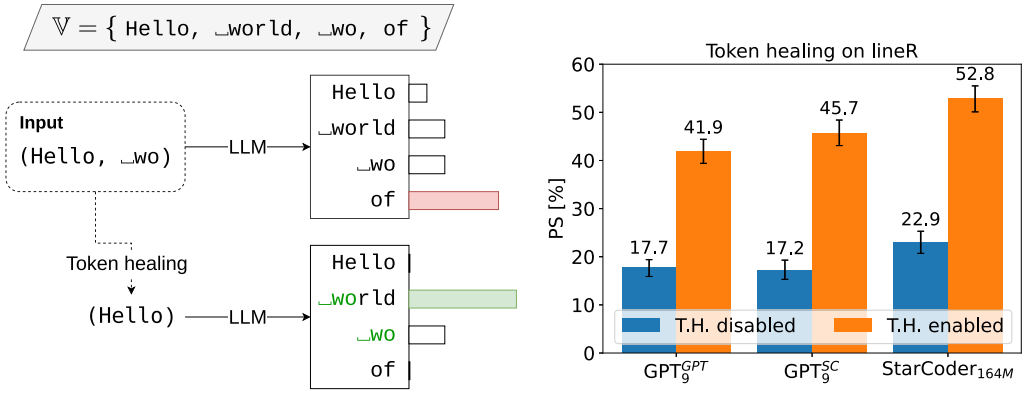


Fig. 1. Illustration of our RETRO approach. The CodeT5+ (Wang et al., 2023b) encoder is used for computing chunk embeddings for retrieval based on the L_2 distance. The color coded arrows indicate which tokens attend to which retrieved chunks – refer to Borgeaud et al. (2022) for more details on the chunked cross-attention (CCA) mechanism. During sampling, the input is padded to be a multiple of the chunk size.

3.2. Token healing

The use of subword tokenization can, in certain cases, lead to undesirable effects during text generation, especially when completing lines of code starting at unnatural word boundaries. Due to the commonly used byte pair encoding (BPE) tokenization algorithm (Sennrich, Haddow, & Birch, 2016), some substrings appear in multiple tokens. Consequently, if the input string s ends with a substring that is the beginning of some token, the output probability distribution for the next token will be biased (Dagan, Synnaeve, & Roziere, 2024). The tokenization of the string s may end with a token that would be merged with the token we are predicting in the BPE algorithm. As a result, tokens that continue the tokenized string s have a higher probability than tokens that would better complete the string s truncated by the last token. In addition, without a corrective measure, called token healing, s may be an out-of-distribution example, leading to unpredictable completion results, since the model has not encountered such unnatural cut-off boundaries during training.

We solve this problem by finding and removing the longest suffix t in the input string $s = s' \cdot t$ that is a prefix of some token $x_i \in \mathbb{V}$, $x_i = t \cdot x_i'$ (Athiwaratkun et al., 2024; Dagan et al., 2024). During decoding, we then constrain the output probability distribution for the next token only to tokens that match the prefix of t. When we generate t in its entirety, we no longer constrain the output distribution. We illustrate the token healing process in Fig. 2(a) and show its practical impact in Fig. 2(b). We observe a significant impact largely due to the choice of the evaluation dataset, which begins code completion at a random position within the line.



(a) Token healing example. (b) Metrics with or without token healing (T.H.).

Fig. 2. Fig. 2(a) shows an example of token healing using a small four token vocabulary. With token healing, the input is rolled back by one token and the output distribution is constrained to tokens that start with ␣wo, improving over the prediction without token healing. In Fig. 2(b), we compare code completion results (without retrieval) with respect to the use of token healing using three different 160 million parameter LLMs that we evaluate in Section 5. The experiments are performed on the evaluation set `lineR` described in Section 4.3. The Prefix Similarity (PS) metric is defined in Section 4.7 (higher values are better).

3.3. In-context RAG

A simpler alternative for retrieval-augmented code completion, compared to RETRO, is to include similar code snippets directly into the model's context. This eliminates the additional complexity of the RETRO architecture, as the In-context RAG method can be used with any autoregressive LLM (Lu et al., 2022; Ram et al., 2023; Zhang et al., 2023b).

Let $x = (x_1, \ldots, x_n) \in \mathbb{V}^n$ be the input sequence of tokens. From x, we construct a query q from the last $m \in \mathbb{N}$ tokens, $q = (x_{n-m+1}, \ldots, x_n)$, and use it to search for the k nearest code snippets from the database $\mathcal{D}$. The tokens of the found snippets $\text{RET}_{\mathcal{D}}(q) = (r_1, \ldots, r_k)$ are concatenated into a sequence of tokens $r = r_1 \cdot \ldots \cdot r_k$. The new context for the model is then $x' = r \cdot x$, which is used to predict the next tokens. The process is illustrated in Fig. 3.

Individual retrieved snippets r_i can be further enhanced, for example by adding metadata about the original file, such as $r_i' =$ `# path/to/file.py\n` $\cdot r_i$. Unlike RETRO, this approach is not limited to retrieving fixed size chunks. We can also dynamically adjust the number of retrieved snippets k based on the remaining size of the input context. The retrieval database $\mathcal{D}$ and the query q need not be constructed based on fixed-length chunks of tokens; instead, the data can be chunked based on lines or logical code parts, such as functions. In our experiments, to make a fair comparison, we use fixed chunks as in the RETRO approach. As with RETRO, the retriever $\text{RET}_{\mathcal{D}}$ only returns examples that are not from the same file as x.

As with the RETRO approach, the retriever $\text{RET}_{\mathcal{D}}(q)$ could operate based on the distance between the embedding of the query and chunks from the database $r_i \in \mathcal{D}$, $d(\mathcal{M}(r_i), \mathcal{M}(q))$. We decided on a simpler approach that works solely based on the Jaccard similarity of tokens and not on embeddings. The Jaccard similarity coefficient J is used over the set Q of tokens from the query q and the set R of tokens from the retrieved snippet r_i,

$$J(Q, R) = \frac{|Q \cap R|}{|Q \cup R|}.$$

The retriever $\text{RET}_{\mathcal{D}}$ thus finds code snippets that have the highest similarity between the query q and the chunks in the database $\mathcal{D}$. We prefer this approach because it does not require an additional model to compute embeddings, making it more suitable for efficient execution on limited hardware and with local projects.

4. Experimental settings

In this section, we present the datasets, evaluation metrics, and experimental settings used to verify if retrieval-augmented code completion can enhance the performance of LLMs when working on local projects. Since we aim to run models on local hardware, we focus on smaller models and efficient retrieval and text generation methods. We assess the impact of retrieval by comparing GPT-2, RETRO, and the In-context RAG approach that incorporates found snippets into the context

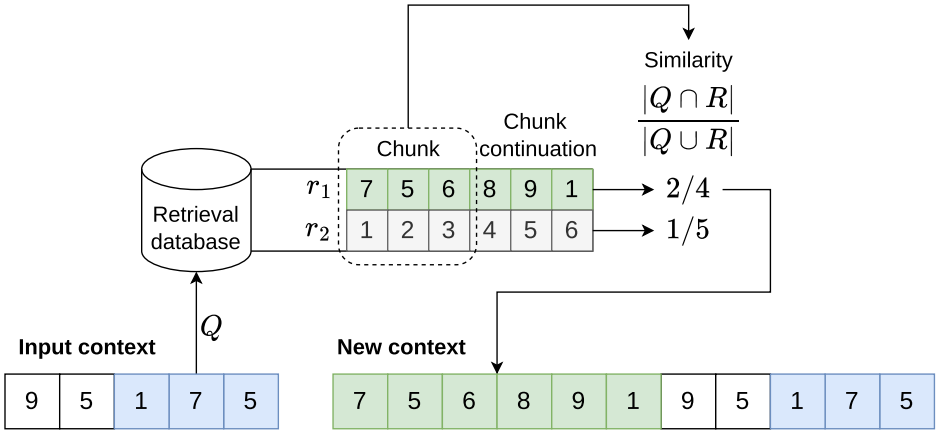


Fig. 3. Our In-context RAG approach based on the Jaccard similarity of chunks of tokens. The retrieved snippet contains continuation tokens and additional metadata about the origin file, while the key for comparing with the query is a fixed length chunk.

Table 1
Tokenizations of the string `def foo(): \n␣␣␣␣pass`. Tokens are comma separated. The StarCoder tokenization requires fewer tokens due to the merging of whitespace characters into a single token.

Tokenization	Tokens
GPT-2	(def, ␣foo, ():, \n, ␣, ␣, ␣, ␣pass)
StarCoder	(def, ␣foo, ():, \n␣␣␣␣, ␣pass)

Table 2
Size of the training and test sets composed from the Python part of the StarCoder dataset.

Split	# Files	# Projects	Total file size
Training set	11 579 885	1 510 155	56 GiB
Test set	1 286 653	168 449	6 GiB
Total	12 866 538	1 678 604	62 GiB

of models without modifying the model architecture or training process. We split the section into seven parts: tokenization, training and evaluation dataset descriptions, retrieval database, model architecture with hyperparameters, training of models, and used comparison metrics. The results are reported in Section 5.

4.1. Tokenization

We compare two subword tokenization methods based on the BPE algorithm (Sennrich et al., 2016) with differing vocabularies. The first version of the vocabulary with 50 257 tokens was trained on a diverse textual dataset obtained from web sources used in the GPT-2 paper (Radford et al., 2019). The second version of the vocabulary with 49 152 tokens was trained on a collection of open-source code from various programming languages reported in the StarCoder paper (Li et al., 2023). Since we work on the code completion task, the use of tokenization from the StarCoder paper shall be more appropriate than using a vocabulary more suitable for natural language processing.

The main drawback of using the text-based GPT-2 vocabulary for code completion is an inefficient handling of whitespaces. In most cases, each space in a longer sequence of spaces is represented by its own token, which significantly reduces the number of useful tokens available in the model's context during the self-attention computation. An example is in Table 1. In addition to less useful data in the model's context, the inefficient tokenization of consecutive spaces also affects the amount of information in each chunk of the retrieval database, as the database is built with a fixed number of tokens per chunk.

4.2. The StarCoder dataset

For training our models, we use the StarCoder dataset[3] (Li et al., 2023). The dataset contains over 300 million open-source files from more than 80 programming languages, but we use only files in the Python programming language. We split the dataset into a training and test set in ratio 90 %; 10 %. We split the data at the project level and not at the file level, so that all files belonging to a project are placed in the same training or test set. The validation set is constructed from 1 % of the test set and is used to monitor metrics during model training to detect any potential overfitting (which we did not observe). After training, we evaluate the models' performance on both the test set and the evaluation sets as described in Section 4.3. Table 2 summarizes the sizes of the training and test sets.

4.3. The RepoEval dataset

For the evaluation of retrieval-augmented line completion on real projects, we use the RepoEval dataset (Zhang et al., 2023b), which contains eight open-source projects from various domains and is divided into three evaluation sets: `line`, `api`, and `function`. The `line` set contains randomly selected individual lines, while the `api` set contains lines with application programming interface (API) calls to objects defined within the project. These calls can be longer than a single line (10 % of examples contain at least 5 lines). The `function` set is intended for completing function bodies, but we do not use it as it requires completing up to 30 lines and evaluation is performed by executing the projects' unit tests.

In both cases, `line` and `api`, the goal is to predict the entire next line or function call. We want to use our models in a code editor even when the cursor is in the middle of a line, while the user is typing. Therefore, we also evaluate the models' performance on `lineR` and `apiR`, where we use the model to complete code from a random position within the line to the end of the original example. More details on `lineR` and `apiR` can be found in Appendix B and an example in Fig. 4.

When evaluating LLMs, there is often the issue of the evaluation set being present in the training set, as training sets are frequently obtained by scraping massive amounts of data from the web. Therefore, we check the overlap between RepoEval and our StarCoder training dataset and find that three of the eight projects in RepoEval are also present in the training set (the project with the most overlap is *alibaba/FederatedScope*, with 48 % of its files contained in StarCoder). In the resulting `line'`, `lineR'`, `api'`, and `apiR'` datasets, we remove these three projects that overlap with the training set.

When simulating and evaluating line completion using the RepoEval dataset, the input context for models without retrieval contains only the lines before the target line within a single file. With retrieval, the retrieval database is built only from individual projects (we do not use the entire StarCoder training set). In our experiments, all text is generated using greedy decoding to achieve determinism and faster execution when completing short lines of code.

4.4. Retrieval database

To train the RETRO approach, we need a database to search for the k-nearest chunks of each input chunk of tokens. In line with Borgeaud et al. (2022) and Wang et al. (2023a), we use the training dataset as the retrieval database. This ensures a fair comparison between RETRO,

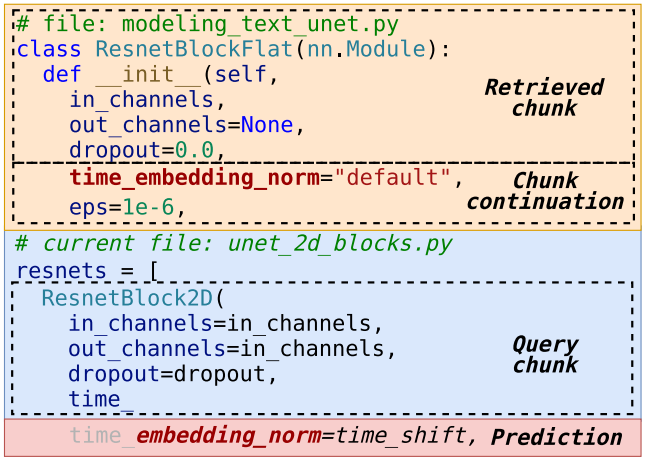


Fig. 4. An example of the In-context RAG approach described in Fig. 3 on our `lineR` subset of RepoEval starting completion from a random position in the target line. The example highlights the importance of including chunk continuation tokens along with the retrieved chunk.

[3] Available at https://huggingface.co/datasets/bigcode/starcoderdata.

which uses retrieval during training, and GPT-2, which does not. Using any other additional dataset, RETRO would have an advantage, as it would have access to a larger amount of data during training.

For building the retrieval database, we use a chunk size of $m = 64$ tokens and the CodeT5+ (Wang et al., 2023b) encoder for computing chunk embeddings. To enable faster retrieval of the k-nearest chunks, we index the stored embeddings with Faiss (Johnson, Douze, & Jégou, 2021) as described in Appendix D. The pretrained CodeT5+ encoder has 110 million parameters, generates normalized 256-dimensional embeddings and was trained on various code tasks. We also experimented with using the BERT-large encoder with 340 million parameters and 1024-dimensional vector embeddings, as done by Borgeaud et al. (2022) and described in Appendix C. However, we opt for the CodeT5+ encoder since it has fewer parameters, outputs smaller dimensional embeddings, and is trained for code understanding, making it a better fit for our use case.

4.5. Model architecture

In accordance with Borgeaud et al. (2022) and Wang et al. (2023a), we compare GPT-2 with RETRO, keeping the architecture of both models identical except that RETRO includes both an encoder and decoder compared to GPT-2 which is a decoder only model. Table 3 summarizes the number of parameters in our models as well as the number of decoder layers. To compare models with roughly equal number of parameters, we also compare GPT-2 with 9 decoder layers to RETRO with 6 decoder layers; this comparison shall ensure that any potential benefit of RETRO does not arise simply from increasing the number of parameters when adding the chunk encoder.

Hyperparameters. We set the hidden dimension d to 1024, and the number of heads in the multi-headed attention to 16, so that each head processes vectors of dimension 64. The number of decoder layers is shown in Table 3. RETRO layers with the retrieval capability are placed at the sixth layer and then at every subsequent third layer (e.g., layers $6, 9, 12$ if there are 12 layers). The encoder in RETRO has two layers. The maximum length of the input context is limited to 384 tokens as in Semenkin, Sokolov, and Vu (2024). Instead of fixed absolute positional embeddings, we use relative positional embeddings RoPE (Su et al., 2024) with a rotational factor of 0.5. For the nonlinear function within fully connected feed-forward networks, we use the SwiGLU function, which has been shown to contribute to better learning outcomes in language modeling tasks (Shazeer, 2020; Touvron et al., 2023). We use shared weights for the input and output embedding tables to reduce the total number of parameters (Press & Wolf, 2017; Vaswani et al., 2017).

4.6. Model training

All training hyperparameters are the same for both models, GPT-2 and RETRO. For optimization, we use the Adam algorithm (Kingma & Ba, 2015) with $\beta_1 = 0.9$ and $\beta_2 = 0.95$. The learning rate is set to 6×10^{-4} and is reduced using a cosine scheduler by a factor of 10 by the end of training. For regularization, we use dropout layers with a probability of 0.1, and the weight decay factor is 0.01. Table 4 summarizes the number of training iterations. For comparing models of all sizes (6, 9, and 12 layers), we conduct training on a smaller number of iterations. For comparing the final models of sizes 6 and 9 layers, we conduct training on the larger number of iterations.

Table 3
The number of parameters of our GPT-2 and RETRO models when using GPT-2 tokenization. The relative change in the number of parameters is shown in parentheses. When using the StarCoder tokenization, the number of parameters for all models decreases by a constant two million parameters due a smaller vocabulary, which affects the number of parameters in the transformer's embedding table.

# Decoder layers	GPT-2	RETRO
6	126 M	**160 M** (+30 %)
9	**164 M**	/
12	201 M	243 M (+21 %)

Table 4
The number of training iterations. Training on eight A100 40 GB GPUs took 2 hours for the smaller number of iterations and 38 hours for the larger number of iterations.

Tokenization	Batch size	# Iterations	# Tokens
GPT-2	384	20 312	3.0×10^9
GPT-2	496	379 032	72.2×10^9
StarCoder	384	15 234	2.2×10^9
StarCoder	512	367 187	72.2×10^9

Implementation details. Our implementation of RETRO and GPT-2 follows the open-source implementation of both models by Wang et al. (2023a), which is in turn based on the original work by Borgeaud et al. (2022). For model training, we use the Megatron-LM framework, which is optimized for training LLMs on multiple GPUs that can be distributed across multiple compute nodes (Narayanan et al., 2021). In our case, all model training and data processing are conducted on a compute node with eight A100 40 GB GPUs. Model training and execution on GPUs is performed using the *bf16* floating-point format.

4.7. Metrics

For evaluating LLMs, we use different metrics for single-token and multi-token predictions. Single-token metrics are used to monitor model training because they are simple and efficient to compute. To assess multi-token predictions, we compare the generated output with the target output.

4.7.1. Single-token metrics

During model training, we use perplexity (PPL), recall-at-k ($\mathrm{R_k}$) and mean reciprocal rank (MRR_k) metrics (Jurafsky & Martin, 2009; Zhang, Lipton, Li, & Smola, 2023a). While PPL is a widespread metric, we find that recall-based metrics provide more intuitive interpretations of results compared to PPL.

Let $x = (x_1, \ldots, x_n) \in \mathbb{V}^n$ be a sequence of tokens. Perplexity $\mathrm{PPL} \in [1, \infty)$ measures the model's uncertainty about the data and is defined as

$$\mathrm{PPL}(x) = \exp\{-\frac{1}{n}\sum_{t=1}^{n} \log P_\theta\left(x_t \mid x_{t-1}, \ldots, x_1\right)\},$$

where $\log P_\theta\left(x_t \mid x_{t-1}, \ldots, x_1\right)$ is the log-probability for token x_t, as predicted by the model with parameters θ. We aim to minimize PPL to its optimal value of 1, which is achieved when the model correctly predicts each token with a probability of 1.

For easier interpretation, we also use recall metrics that summarize the probability that the predicted token is one of the correct tokens. Let $\hat{y}_i \in \mathbb{R}^{|\mathbb{V}|}$ be the output distribution of the next token after x_i, sorted by descending probability (i.e., the first element of $\hat{y}_i$ corresponds to the most probable next token). Then, rank_i is the index in $\hat{y}_i$ that corresponds to the next token x_{i+1}.

The recall-at-k metric, $\mathrm{R_k} \in [0, 1]$, equals 1 for x_i if x_{i+1} is among the k most probable suggested tokens. For the entire x, we take the average

$$\mathrm{R_k} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}\left(\mathrm{rank}_i \le k\right).$$

$\mathrm{R_k}$ can be interpreted as the probability that the next token is among the k most probable predicted tokens.

The Mean Reciprocal Rank $\mathrm{MRR}_k \in [0,1]$ is defined as

$$\mathrm{MRR}_k = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{\mathrm{rank}_i},$$

where $\frac{1}{\mathrm{rank}_i}$ equals 0 if $\mathrm{rank}_i > k$. We aim to maximize $\mathrm{R_k}$ and MRR_k to their optimal value of 1. When comparing single-token metric results, we must be cautious when comparing results obtained with different tokenizations, as the size and content of the vocabularies affect the obtained values.

4.7.2. Multi-token predictions

For multi-token predictions, we evaluate the quality of code completion, not only based on the prediction of the next token, but on generating multiple tokens. We calculate metrics on *strings*, removing leading and trailing whitespace from the strings (Lu et al., 2022; Zhang et al., 2023b).

Let s be the model's prediction and t the target string. Exact Match $\mathrm{EM}(s,t) \in \{0,1\}$ is 1 if the strings s and t match exactly and 0 otherwise. Edit Similarity $\mathrm{ES}(s,t) \in [0,1]$ considers how many edits are needed to transform s into t:

$$\mathrm{ES}(s,t) = 1 - \frac{\mathrm{lev}(s,t)}{\max(|s|,|t|)},$$

where lev is the Levenshtein distance (Levenshtein et al., 1966). When combining multiple pairs (s,t) from a sample of predicted and target results $\mathcal{Y}$ for EM and ES, we calculate the average, $\mathrm{EM} = \frac{1}{n}\sum_{(s,t)\in\mathcal{Y}}\mathrm{EM}(s,t)$ and similarly $\mathrm{ES} = \frac{1}{n}\sum_{(s,t)\in\mathcal{Y}}\mathrm{ES}(s,t)$.

Prefix Similarity $\mathrm{PS}(s,t) \in [0,1]$ measures the prefix matching of characters in the strings s and t. For $\mathrm{PS} \in [0,1]$, we take into account that lines of code have different lengths when combining multiple examples:

$$\mathrm{PS} = \frac{\sum_{(s,t)\in\mathcal{Y}}|\pi(s,t)|}{\sum_{(\cdot,t)\in\mathcal{Y}}|t|},$$

where $\pi(s,t)$ denotes the longest common prefix of s and t. We aim to maximize all three metrics EM, ES, and PS to their optimal value of 1.

When reporting multi-token prediction metrics, we calculate 95% confidence intervals obtained using the BCa (Efron, 1987) bootstrap method with 1000 samples. The notation a_b^c means that a is the calculated metric on the sample, and b and c are the endpoints of the confidence interval $[b,c]$.

5. Results

In this section, we report the results of experiments outlined in Section 4. We start by comparing the performance of RETRO and GPT-2 on the StarCoder dataset, followed by the evaluation on the RepoEval dataset, which simulates retrieval from local projects. In this setting, we also compare the In-context RAG approach. We finish the section with a qualitative analysis of the results.

5.1. Evaluation on the StarCoder test set

In this section, we present the results of our trained models on the test set from Section 4.2. We are interested in the contribution of retrieval with RETRO compared to the baseline GPT-2 model. We first examine the impact of the retrieval database size, tokenization, and the number of parameters on model performance. We also compare retrieval from the training and test set simulating different numbers of projects in a local database.

First, we examine the results of single-token metrics on the test set. When reporting metrics for RETRO, the retrieval database is built from the union of the training and test sets, while during training only the training set is used (Borgeaud et al., 2022; Wang et al., 2023a). If retrieval from the test set was forbidden, RETRO would not be able to find related code snippets from the same project as the test example, due to the project-level split into training and test sets. We also report metrics calculated starting from the end of the first RETRO chunk $x^{\geq 64} = (x_{64}, \ldots, x_n)$ instead of the entire input sequence $x = (x_1, \ldots, x_n)$. For the first $m - 1 = 63$ tokens, RETRO does not use retrieval and therefore functions like the baseline model. The notation $\mathrm{GPT}_6^{\mathrm{GPT}}$ refers to the GPT-2 model with 6 layers and GPT-2 tokenization, while $\mathrm{GPT}_6^{\mathrm{SC}}$ refers to the using the model with StarCoder tokenization (and similarly $\mathrm{RETRO}_6^{\mathrm{GPT}}$ for RETRO).

In Fig. 5 we compare our models of all sizes trained on a smaller number of iterations and in Table 5 we report the single-token metrics of our smaller sized models trained over more iterations. In all cases we observe that the single-token metrics for RETRO are better than those for GPT-2 of comparable size. However, the difference between the two models decreases when comparing models with approximately equal number of parameters, i.e. RETRO with 6 layers to GPT-2 with 9 layers, as seen in Table 5. In Table 5 we observe that the differences between RETRO and GPT-2 are higher when calculating metrics from token 64 onwards, suggesting the usefulness of the retrieval mechanism. Increasing the number of layers and thus the number of model parameters also contributes to improvements in metrics, as is evident from Fig. 5. However, by training smaller models over more iterations, we achieve better metrics than using larger models trained for fewer iterations. The relative differences are larger in bigger models, which may be due to the use of more RETRO layers, as well as the effect of 21 % more parameters in $\mathrm{RETRO}_{12}^{\mathrm{GPT}}$ compared to $\mathrm{GPT}_{12}^{\mathrm{GPT}}$.

We hypothesize that the significant differences in single-token metrics between GPT-2 and StarCoder tokenization are due to the different vocabularies making direct comparison of results less meaningful, so we should instead compare relative differences. Nevertheless, we surmise that GPT-2 tokenization achieves better single-token metrics in Fig. 5 and Table 5 due to the whitespace issue from Table 1, as the model gets "easy points" by predicting individual spaces. Despite larger absolute differences in individual metrics between GPT-2 and StarCoder tokenization, the relative differences in Fig. 5 and in Table 5 are of comparable magnitude, suggesting a similar contribution of RETRO regardless of the tokenization used.

Table 5
Single-token metrics on the test set with models trained on a **larger** number of iterations (see Table 4). Reported are also the relative differences of metrics compared to the baseline GPT-2 models.

					Offset $x^{\geq 64}$			
Model	PPL	R_1	R_5	MRR_5	PPL	R_1	R_5	MRR_5
$\mathrm{GPT}_6^{\mathrm{GPT}}$	2.708	79.27	90.25	83.74	2.445	80.97	91.41	85.25
$\mathrm{GPT}_9^{\mathrm{GPT}}$	2.602	79.95	90.69	84.33	2.347	81.67	91.85	85.84
	-3.91 %	0.85 %	0.49 %	0.70 %	-4.01 %	0.86 %	0.48 %	0.69 %
$\mathrm{RETRO}_6^{\mathrm{GPT}}$	2.536	80.40	91.11	84.78	2.257	82.23	92.46	86.50
	-6.35 %	**1.43 %**	**0.95 %**	**1.24 %**	**-7.69 %**	**1.56 %**	**1.19 %**	**1.47 %**
$\mathrm{GPT}_9^{\mathrm{SC}}$	3.946	70.66	86.54	77.07	3.433	73.06	88.18	79.19
$\mathrm{RETRO}_6^{\mathrm{SC}}$	3.855	70.98	86.98	77.45	3.293	73.65	88.87	79.84
	-2.30 %	**0.45 %**	**0.51 %**	**0.49 %**	**-4.08 %**	**0.81 %**	**0.78 %**	**0.82 %**

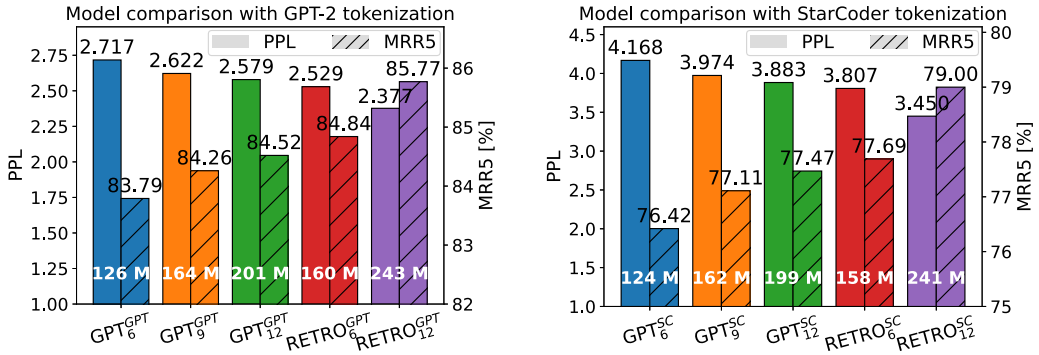


Fig. 5. PPL and MRR_5 metrics on the test set with models trained on a **smaller** number of iterations (see Table 4) with *GPT-2* tokenization on the left and *StarCoder* tokenization on the right. For each model, the left bar shows PPL and the right bar shows MRR_5. Reported metrics are calculated on the offset input $x^{\geq 64}$. The values of the left and right results should not be directly compared due to different tokenizations. The figures also show the models' sizes.

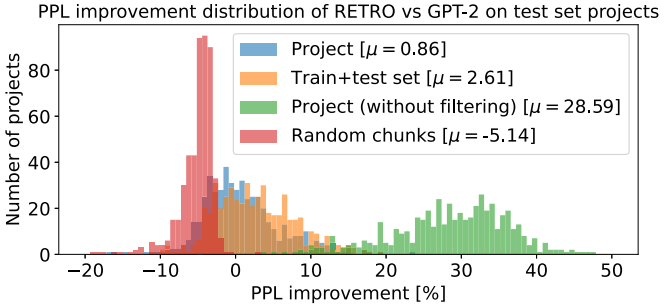


Fig. 6. Distribution of relative improvement in PPL between $RETRO_6^{GPT}$ and GPT_9^{GPT} on 500 projects from our StarCoder test set based on the used retrieval database.

Table 6
Comparison of the relative improvement of single-token metrics between $RETRO_6^{GPT}$ in GPT_9^{GPT} on 500 projects from the test set with respect to the used retrieval database. The metrics are in percentages, contain 95-percent confidence intervals and are calculated on the offset input $x^{\geq 64}$.

Retrieval database	ΔPPL	ΔR_1	ΔR_5	ΔMRR_5
Project	$0.86^{1.36}_{0.42}$	$0.12^{0.23}_{0.01}$	$0.24^{0.31}_{0.17}$	$0.18^{0.27}_{0.09}$
Training set	$-0.05^{+0.37}_{-0.45}$	$-0.03^{+0.07}_{-0.12}$	$0.12^{0.18}_{0.06}$	$0.05^{+0.13}_{-0.03}$
Training + test set	$\mathbf{2.61}^{3.04}_{2.15}$	$\mathbf{0.53}^{0.64}_{0.43}$	$\mathbf{0.51}^{0.58}_{0.44}$	$\mathbf{0.54}^{0.63}_{0.45}$
Project *(without filtering)*	$28.59^{29.39}_{27.89}$	$7.19^{7.44}_{6.99}$	$4.78^{4.97}_{4.64}$	$6.24^{6.45}_{6.06}$
Random chunks	$-5.14^{-4.94}_{-5.39}$	$-1.10^{-1.05}_{-1.17}$	$-0.61^{-0.58}_{-0.64}$	$-0.89^{-0.85}_{-0.94}$

Projects in a local database. For practical local use of RETRO, a small number of parameters would not be beneficial if we needed a retrieval database with a billion tokens to see benefits. Therefore, we examine the contribution of RETRO using retrieval databases constructed from *individual* projects in the test set by sampling 500 projects, such that the projects are evenly distributed with respect to the number of files, up to a maximum of 300 files.

In Fig. 6, we compare the relative improvement of $RETRO_6^{GPT}$ against GPT_9^{GPT} on projects using different retrieval databases. We limit the comparison to RETRO with 6 layers and GPT-2 with 9 layers to reduce the impact of the difference in the model sizes. Retrieving from the entire StarCoder training set contributes to a greater improvement compared to retrieving only from individual projects, which is consistent with the findings of Borgeaud et al. (2022), who observe improvements with increasing the retrieval database size. We also test retrieval from the project without removing snippets originating from the same file as the input context, which gives an upper bound estimate for RETRO since the found snippets contain the correct next tokens. As expected, the difference with GPT-2 is in this case more significant, but it does not reach a perfect success rate. A lower bound estimate for RETRO is obtained when we retrieve random chunks from the entire training set. In this case, we observe that the 6-layer $RETRO_6^{GPT}$ performs worse than the 9-layer GPT_9^{GPT}, which is expected, as it does not benefit from retrieval and consequently behaves as a 6-layer GPT-2 with added noise.

From Table 6, we see that the single-token metrics using only $RETRO_6^{GPT}$ with the training set (without the test set) for retrieval are comparable to those of GPT_9^{GPT}. This can be attributed to the fact that retrieving from the training set does not provide new information, as both models have seen the entire training set during training. We therefore conclude that retrieval is only meaningful on new, previously unseen data for the model. When retrieving from projects, the data is new, as the projects are sampled from the test set. An implication of this observation is that it might be beneficial to train RETRO using a retrieval database different from the training set. The training could also include examples of random chunks to help the model better learn to skip irrelevant information, and examples of same-document snippets to help the model better learn to copy correct answers.

5.2. Completing code lines from local projects with RepoEval

In this section, we aim to simulate and evaluate completing lines in local projects using the RepoEval dataset, as described in Section 4.3. We first present the results of RETRO, GPT-2, and In-context RAG in this setting, followed by an experiment with larger models, and detailed analysis of our most successful approach, namely In-context RAG. We end the section with an analysis of the impact the quality of the retrieved snippets has on the code-completion performance.

5.2.1. RETRO and GPT-2 results on local projects

In Fig. 7, we compare our trained models, RETRO and GPT-2, using both GPT-2 and StarCoder tokenizations. The metrics confirm the advantage of StarCoder tokenization, which better handles the whitespace issue mentioned in Section 4.1. Similar to the single-token metrics, with multi-token predictions, we observe that the 6-layer RETRO achieves significantly better performance than the 6-layer GPT-2. However, the difference between the 9-layer GPT-2 and the 6-layer RETRO is not as pronounced, especially with the StarCoder tokenization.

Metrics on `lineR` are higher than on `line` (similarly for `apiR` and `api`), because in the case of `line` we predict entire lines, while in `lineR` the model already has some additional context about the content of the current line. This is important because predicting the entire line requires correctly predicting the start of the line; otherwise, the EM and PS metrics are immediately zero. Predicting the correct start of a line is not always straightforward, as multiple beginnings can make sense.

In Fig. 7, we also report the metrics for the In-context RAG approach (more in Section 5.2.3). Despite its simplicity, the contribution of the In-context RAG is more significant than the use of RETRO. When combining In-context RAG with RETRO, we observe smaller improvements

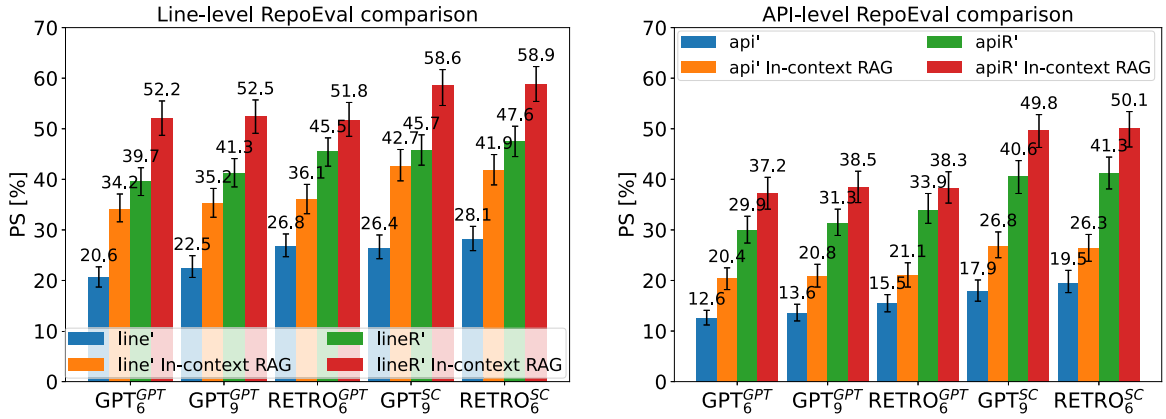


Fig. 7. Comparison of our models using GPT-2 and StarCoder tokenization on the RepoEval line-level (left) and API-level (right) datasets without overlap with the training set. The bar plots contain 95-percent confidence intervals of the prefix similarity metric.

compared to enhancing GPT-2 with In-context RAG. A possible explanation is that including retrieved snippets into the input context "confuses" RETRO's chunk based retrieval, which now finds neighbors of the already retrieved snippets for part of the context. Additionally, snippets included in the context receive the same treatment as other input tokens through processing in the transformer, while RETRO incorporates the retrieved chunks using cross-attention only in the sparsely dispersed RETRO layers.

5.2.2. Comparison with larger models

Using small models is beneficial for local model execution, but one of the advantages of LLMs is their ability to scale by increasing the number of parameters (Kaplan et al., 2020). In Fig. 8 and Table 7, we examine the contribution of larger models compared to our smaller models. The open-source models StarCoder$_{164M}$ (with 164×10^6 parameters) and StarCoder$_{15.5B}$ (with 15.5×10^9 parameters) are trained on the entire StarCoder dataset and further fine-tuned on Python (Li et al., 2023). StarCoder$_{164M}$ serves as a good comparison to our models due to its comparable number of parameters and for assessing the contribution of training on a larger dataset with a larger context size (8192 compared to our size of 384).

Increasing the number of parameters in some cases more than doubles the performance compared to smaller models, as seen in Table 7. However, with the use of In-context RAG, even smaller models can achieve results comparable to larger models (without retrieval). The positive impact of retrieval is not limited to smaller models, as incorporating useful information from the project into the context benefits the completions of larger models as well.

Results on `api` are lower than on `line` due to completing function calls from projects and also due to the longer length of examples, which can contain more than a single line. Consequently, the model must correctly predict more tokens, increasing the likelihood of error. For longer examples the differences are even greater due to the use of greedy decoding and the propagation of errors, which are more frequent in smaller models.

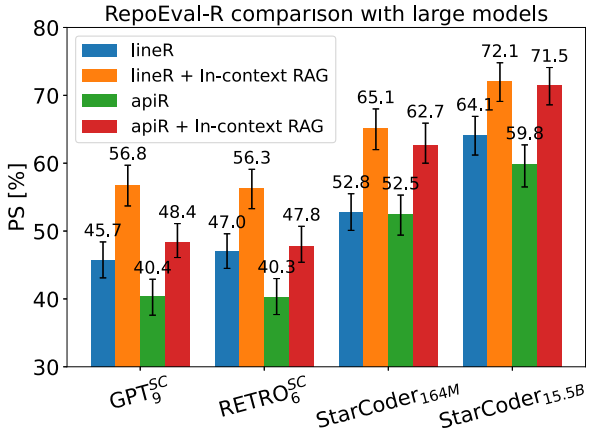


Fig. 8. Comparison of our models with larger models on the line-level and API-level RepoEval datasets starting completion from random positions. The bar plots contain 95-percent confidence intervals of the prefix similarity metric.

5.2.3. In-context RAG

Since the found snippets are included directly in the model's context, it is important that the model supports a sufficiently large context to append the retrieved data in addition to the input context. When using our models with a context size of 384 tokens, we often need to truncate the input sequence to leave enough space to include the retrieved snippets at the beginning of the context. For our models, GPT-2 and RETRO, we include only one retrieved snippet due to the small context size of the model. For models with larger context sizes, we include two retrieved snippets. In Table 7 and in Figs. 7 and 8, we report multi-token prediction metrics both with and without retrieval as described in Section 3.3. The results indicate that using In-context RAG leads to improvements in multi-token predictions compared to the baseline models in all cases.

Copying. In Section 5.1, we estimate the upper bound of single-token metrics for RETRO by not removing chunks retrieved from the same file as the input context. Similarly, we want to determine the upper bound of model performance when using In-context RAG if the retrieved snippets can contain the continuation of the current file we are completing. This means that the context includes the line we want to complete, allowing the model to simply copy it from the context. In Fig. 9, we see that copying from retrieved snippets is very effective, while copying with RETRO without In-context RAG performs similarly to using RETRO with In-context RAG without copying.

In addition to the explanations in Section 5.1, we also note that because the first RETRO layer is the sixth layer (which is also the last and only RETRO layer for RETRO$_6^{SC}$), certain information from the input tokens may get diluted, preventing direct copying of tokens from the encoded chunks with chunked cross-attention. We hypothesize that adding copying examples to the training would improve the model's ability to copy, and RETRO layers could be included closer to the beginning of processing instead of only in the sixth layer. Accurate copying is important because when completing code, we want to extract not only the semantic meaning from the found snippets but also exact facts (e.g., the exact name of an existing function and not a hallucinated but semantically similar name).

Table 7
Results of comparing smaller and larger models on RepoEval. The results for code-davinci-002 are from Zhang et al. (2023b).

					In-context RAG		
Model	Data	EM	ES	PS	EM	ES	PS
GPT_9^{SC}	line	$18.0^{19.9}_{16.1}$	$47.0^{48.4}_{45.3}$	$23.2^{25.0}_{21.4}$	$33.9^{36.0}_{31.6}$	$58.4^{60.2}_{56.7}$	$40.9^{43.6}_{38.4}$
	api	$8.8^{10.1}_{7.3}$	$39.3^{40.6}_{37.8}$	$15.0^{16.3}_{13.5}$	$21.1^{23.2}_{19.1}$	$51.7^{53.3}_{50.0}$	$26.9^{29.0}_{24.9}$
$RETRO_6^{SC}$	line	$19.6^{21.8}_{17.7}$	$50.1^{51.8}_{48.6}$	$26.4^{28.1}_{24.5}$	$31.6^{33.8}_{29.2}$	$57.2^{58.9}_{55.6}$	$39.5^{42.2}_{37.3}$
	api	$9.6^{11.0}_{8.1}$	$42.7^{44.1}_{41.3}$	$16.8^{18.2}_{15.4}$	$20.1^{21.9}_{18.1}$	$50.5^{51.9}_{48.6}$	$26.3^{28.3}_{24.4}$
$StarCoder_{164M}$	line	$25.1^{27.0}_{22.9}$	$53.1^{54.7}_{51.4}$	$31.9^{34.2}_{29.9}$	$41.6^{44.0}_{39.1}$	$64.6^{66.3}_{62.6}$	$48.1^{50.9}_{45.2}$
	api	$23.2^{25.1}_{21.0}$	$52.6^{54.1}_{50.9}$	$32.4^{34.6}_{30.2}$	$34.5^{36.8}_{32.2}$	$63.0^{64.6}_{61.4}$	$43.6^{46.0}_{41.4}$
$StarCoder_{15.5B}$	line	$\mathbf{39.4}^{41.8}_{37.1}$	$\mathbf{64.4}^{66.2}_{62.7}$	$\mathbf{44.8}^{47.5}_{42.3}$	$51.0^{53.4}_{48.8}$	$72.2^{73.7}_{70.6}$	$\mathbf{56.5}^{59.2}_{53.8}$
	api	$32.1^{34.2}_{29.8}$	$61.2^{62.9}_{59.4}$	$41.3^{43.8}_{39.0}$	$44.1^{46.4}_{41.5}$	$71.4^{72.9}_{69.8}$	$53.6^{56.1}_{51.2}$
code-davinci-002	line	38.69	62.58	/	**55.94**	**74.34**	/
	api	**32.50**	**61.52**	/	**47.75**	**73.30**	/

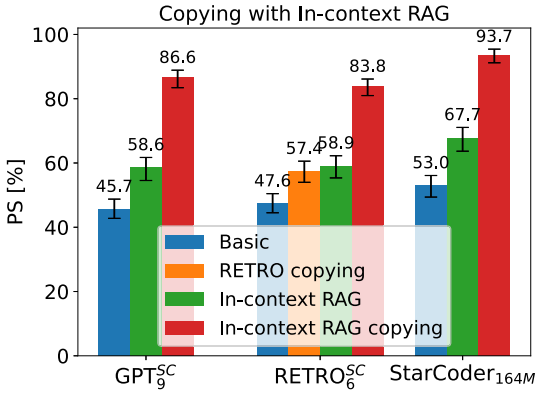


Fig. 9. Comparison of copying abilities using In-context RAG on `lineR'`. With copying, retrieved snippets can be from the same file as the input context, which means including the correct continuation of the current line in the model's context. *RETRO copying* shows the use of RETRO without filtering out the current file from retrieval but without additional In-context RAG (see also Section 5.1). The bars show 95 % confidence intervals for the prefix similarity metric PS.

5.2.4. Impact of retrieval similarity

Intuitively, we expect the benefit of retrieval-augmented line completion to depend on the quality of the retrieved snippets. Therefore, we first examine the impact of the retrieved snippets' similarities on the exact match metric EM with the results shown in Fig. 10. As expected, the metric increases with higher matching according to the Jaccard similarity coefficient. We also find that most of the metric improvement comes from projects where snippets with high similarity to the query from the input context are found. The benefits of retrieval are thus project-dependent, with greater benefits in projects with repetitive code, as a high Jaccard similarity indicates token overlap between files.

Table 8
Proportions of improved and worsened examples based on prefix similarity when comparing the baseline model (rows) with the model with retrieval (columns) on the `lineR` dataset. The first column of the first row tells us that $RETRO_6^{SC}$ completes the line better than GPT_9^{SC} in 12.3 % of cases, worse in 7.9 % of cases, and in the remaining cases, the predictions of both models are the same.

		In-context RAG		
	$RETRO_6^{SC}$	$RETRO_6^{SC}$	GPT_9^{SC}	$StarCoder_{164M}$
GPT_9^{SC}	12.3 / 7.9	20.3 / 9.3	**17.4 / 6.2**	26.2 / 6.3
$RETRO_6^{SC}$	/	**14.5 / 6.0**	17.2 / 8.8	24.4 / 6.6
$StarCoder_{164M}$	11.4 / 15.6	17.5 / 14.6	17.2 / 14.1	**15.8 / 3.8**

Using RETRO or In-context RAG generally improves multi-token predictions. However, there are cases where the model without retrieval correctly completes a line, while the model with retrieval does not, as summarized in Table 8. Most examples remain unchanged regardless of retrieval use, suggesting that retrieval is beneficial only in specific cases. While positive examples dominate, the proportion of worsened results is not negligible and we want to reduce it.

Now we focus only on the examples highlighted in Table 8, where we detect worsening or an improvement in metrics. From Fig. 11, we find that the impact of retrieval is negative mainly at low similarities to the query, while the positive impact is more present at higher similarities. To mitigate the consequences of poor retrieval, we examine the idea of whether it makes sense to limit retrieval to found snippets with

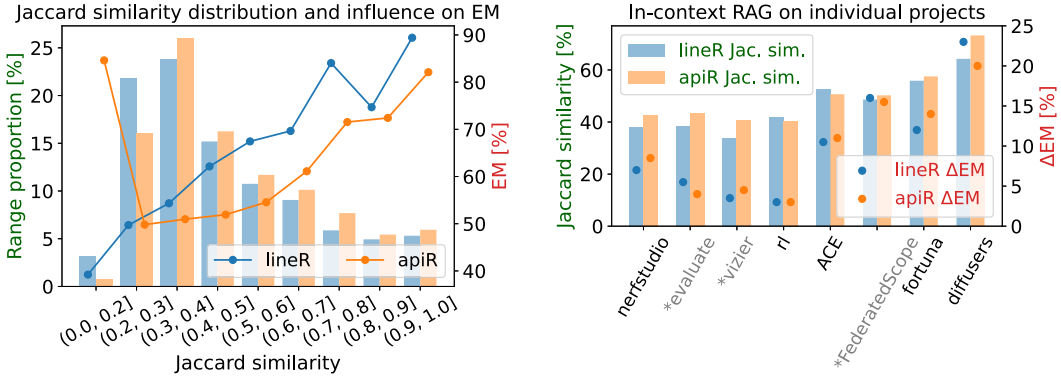


Fig. 10. Impact of the Jaccard similarity coefficient evaluated on `lineR` and `apiR` using the $StarCoder_{164M}$ model and In-context RAG. The left side shows EM increasing as the Jaccard similarity increases, and the distribution of similarities, which is slightly higher on `apiR` than on `lineR`. The right side shows the average Jaccard similarity for each of the eight projects in `lineR` and `apiR` and the difference in EM for each project individually compared to the baseline model without retrieval. Projects with gray names and an asterisk are those removed in `lineR'` and `apiR'` due to a non-empty overlap with the training set.

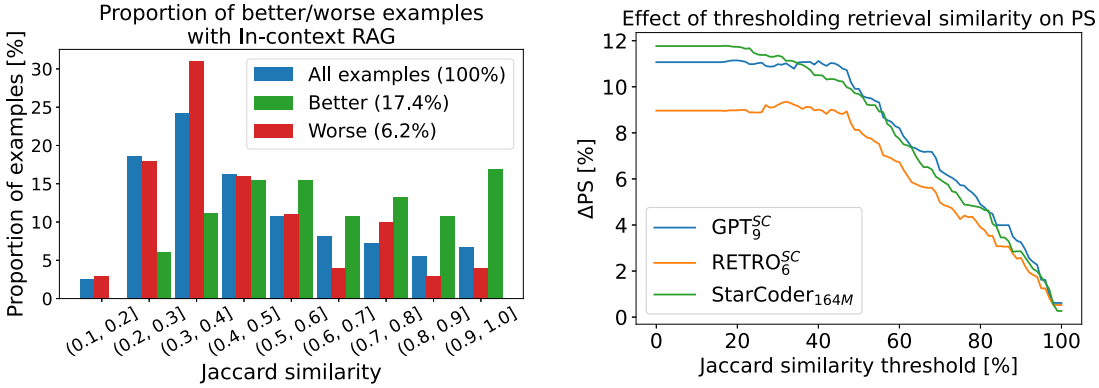


Fig. 11. The left side shows the proportion of cases that are better or worse with In-context RAG compared to the baseline model without retrieval. The model used is GPT$_9^{SC}$ on `lineR` with PS being compared. Retrieval is beneficial mainly when there is a high similarity of the retrieved snippets. The right side shows the difference in the PS metric to the baseline model when limiting retrieval to cases above a certain similarity threshold.

similarities higher than some threshold value. Unfortunately, the right side of Fig. 11 does not show a significant benefit from limiting to sufficiently similar snippets.

5.3. Qualitative analysis

In Section 5.2, we found that retrieval generally benefits model predictions, but there are still cases where retrieval degrades the accuracy of predictions. In this section, we conduct a qualitative review of successful and less successful line completion examples, using our GPT$_9^{SC}$ and RETRO$_6^{SC}$ models.

Common errors. Our small models (with or without retrieval) mostly make semantic errors; syntactic errors are rare. Semantic errors require a higher level of code understanding, which might exceed the capabilities of our small models. Difficulties often arise when completing natural language strings (e.g., comments and documentation within the code), at the beginning of files in `import` statements, and when using boolean or numerical constants. Degradation is also present in longer predictions due to greedy decoding and error propagation. For completing `import` statements, the model would need information about the upcoming lines of the file to infer which modules to import into the program. In completing documentation, we have a similar problem if we are documenting code that has not yet been written and follows in the subsequent lines. Additionally, our string matching based evaluation does not distinguish between semantically equivalent ways of expressing messages causing examples like in Fig. 12 to achieve low metrics. Nevertheless, retrieval can offer an advantage in such cases by providing the model with an example of the existing documentation style.

Hallucinations. Handling hallucinations in code completion is more manageable compared to natural language, as the proposed code can be verified with static code analysis tools to discard suggestions containing errors (e.g., calling a non-existent function or using an incorrect number of arguments). However, static analysis cannot resolve semantic errors, such as incorrect conditions in loops or the improper order of operations. Therefore, careful attention and understanding of the code are necessary when using code completion tools.

```
parser.add_argument(
  "--validation_epochs",
  type=int,                                          Retrieved
  default=50,                                        snippet
  help=(
      "Run dreambooth validation every X epochs. ..."))
parser.add_argument(
  "--validation_epochs",
  type=int,                                          Input
  default=50,                                        context
  help=(
  'Run validation every X epochs.'))                 Actual continuation
  'Number of epochs to run validation for.'))        Without retrieval
  'Run dreambooth validation every X epochs.'))      With retrieval
```

Fig. 12. An example of completing documentation where retrieval guides the model to copy from a highly similar found snippet. The code is shortened for clarity. This example also shows the difficulty of evaluating natural language using string similarity based metrics, as the model's prediction without retrieval is very reasonable.

Advantages and disadvantages of retrieval. As discussed in Section 5.2.4, the advantage of retrieval-augmented completion becomes apparent when highly similar examples to the input context are found in the project. An example of repetitive code occurs when writing tests, which also involves the use of objects defined within the project. In most cases, the model can focus on the relevant information in the context and successfully skip over unhelpful retrieved snippets. However, when the model performs worse with retrieval, it is often due to misleading snippets that conflict with the information of the current file. For example, the model might use a function call as found in a related file instead of how it should be used according to the current file, as seen in Fig. 13.

Evaluation limitations. Evaluating the quality of code completion using metrics is challenging (Evtikhiev, Bogomolov, Sokolov, & Bryksin, 2023). In many cases, a line can be completed in multiple ways, but evaluation datasets typically contain only one correct answer. This problem could be mitigated by sampling multiple generated solutions, thereby increasing the likelihood of a correct match. A better solution would be to execute the generated code using unit tests defined in the project. However, besides the complexity of running code (e.g., setting up a development environment for each project), this is not the most suitable quality measure in our case since we are completing individual lines rather than entire functions. Instead of string based metrics, LLMs could be used to evaluate quality, as they have a higher correlation with human judgements (Liu et al., 2023). Ultimately, it is the developer who decides whether to accept the suggested line, so for evaluation, it would be necessary to ask users for feedback or conduct A/B tests of the final product integrated into a code editor.

6. Conclusion and future work

In this work, we examined the impact of retrieval from local projects on the success of completing lines of code using LLMs. We compared the GPT-2 and RETRO models, which we trained on a dataset of Python files from the StarCoder dataset. We compared two different tokenization approaches and highlighted the importance of token healing and the proper tokenization of whitespace when completing code, as it affects the amount of information in the context and the storage size of tokenized data. We limited GPT-2 and RETRO to a smaller size of around 160 million parameters and reduced the impact of the differing number of parameters when comparing the two models. We confirmed that RETRO with retrieval from the local project improves metrics compared to both the 6-layer and 9-layer GPT-2 on all considered benchmarks.

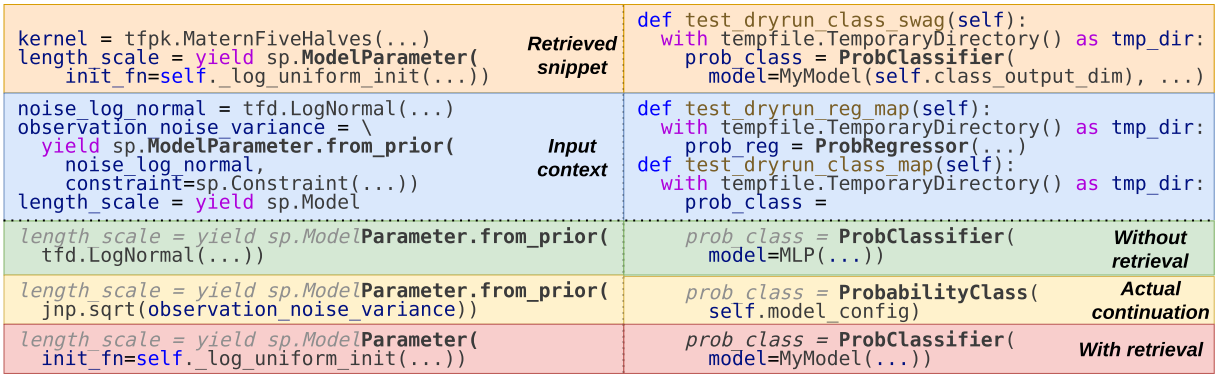


Fig. 13. On the left is an example of misleading retrieval where the model incorrectly uses a function as it appears in the retrieved snippet instead of how it is used in the current file. On the right is an example of completing a test function in a project where a model with a longer context could correctly complete the lines, as the name `ProbClassifier` would have been encountered at the beginning of the file in the `import` statements. However, with a context size of 384, the beginning of the file is lost, but retrieval finds a similar example in other tests in the project, enabling successful completion compared to the baseline model. The code is shortened for clarity in both cases.

However, the difference between the 9-layer GPT-2 and the 6-layer RETRO is not pronounced, indicating the importance of considering the number of model parameters when comparing small models. We further examined the impact of retrieval using In-context RAG, which includes snippets found using the Jaccard similarity of tokens into the context. Using In-context RAG to retrieve from projects in RepoEval improved results of both small and larger models, but the improvements depended on the quality of retrieval and the projects' characteristics. We found that the similarity of the retrieved snippets and the ability to copy from them have a significant impact on the results, but the copying capabilities of the 6-layered RETRO were found to be lacking. Based on our results, we deem that using small language models with In-context RAG is more sensible than using RETRO for the task of completing lines of code with retrieval from local projects due to the simplicity and broader applicability of In-context RAG.

In summary, users looking to apply our work would do well to consider picking the largest model size that fits within their constraints and then apply retrieval from local projects.

Future work. RETRO has proven useful for code completion, but several potential improvements would require retraining the model. Training could include examples that would improve the ability to copy from retrieved snippets and to ignore irrelevant information. For code completion, it would be beneficial to experiment training RETRO with the fill-in-the-middle technique (Donahue, Lee, & Liang, 2020), which reshapes the context to include code after the cursor position. Training RETRO with a longer context could also be enhanced by training on projects instead of individual files. For better integration with In-context RAG, the retrieved snippets could be appended to the input context during training (Wang et al., 2023a). Additionally, it would be worthwhile to explore training and using RETRO with different retrieval methods, such as sparse BM-25 search instead of methods based on dense embeddings.

A major advantage of In-context RAG is that it does not require updating the model weights, but doing so could still be beneficial for improving performance either by fine-tuning the model with included retrieved snippets or by fine-tuning the retriever (Izacard & Grave, 2021; Karpukhin et al., 2020). Instead of retrieving snippets based on the Jaccard similarity of tokens, a future area to investigate is incorporating sparse search with the benefits of dense semantic search and check whether hybrid generative and retrieval-based models could further improve performance. For more efficient execution in code editors, models could be fine-tuned to append to the end of the context instead of the beginning, which would improve the caching of keys and values in self-attention during frequent code editing. A next step would also be to explore the impact of combining In-context RAG with the fill-in-the-middle technique, which is becoming increasingly widespread (Wu et al., 2024).

By training on more data, using appropriate tokenization, and improving the information present in the context with retrieval and fill-in-the-middle, alongside approaches such as weight quantization, we believe that code completion models have great potential for local use in programming tools.

CRediT authorship contribution statement

Marko Hostnik: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft, Writing – review and editing. **Marko Robnik-Šikonja**: Conceptualization, Writing – original draft, Writing – review and editing, Supervision, Project administration.

Funding sources

This research was funded by the Slovenian Research and Innovation Agency (ARIS) core research programme P6-0411 and project GC-0002. The work was also supported by the EU through ERA Chair under Grant 101186647 (AI4DH) and cofinancing for research innovation projects in support of green transition and digitalisation (project PoVeJMo, no. C3.K8.IB) (Marko Robnik-Šikonja).

Data availability

The StarCoder dataset (Li et al., 2023) and the RepoEval (Zhang et al., 2023b) dataset are publicly available.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Marko Hostnik reports a relationship with JetBrains sro that includes: employment. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We thank JetBrains for motivating the problem and providing the computing power for conducting our experiments.

Appendix A. StarCoder dataset analysis

In this paper, we examine the benefit of including retrieved similar code snippets from the project into the code completion process. We hypothesize that finding similar examples can be useful when predicting

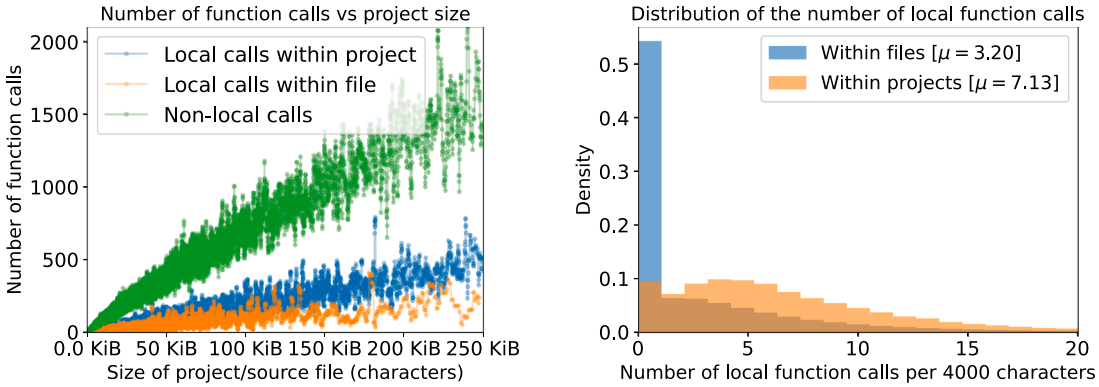


Fig. A.1. Comparison of references to functions defined within projects in the StarCoder dataset. Non-local function calls refer to references to functions not defined within the project (e.g., calls to functions from the standard library). In the right figure, due to varying file lengths, we calculate the average number of calls per 4000 characters. The number of calls increases with project size (left graph), and calls are more frequent when considering the project as a whole compared to individual files (right distribution). In the right distribution, the number of calls within files stands out near zero, which we attribute to calls to functions defined in other project files. Within individual files, such calls are counted as non-local.

symbols (e.g., function names, variables, classes) defined within the local project, as such names are not present in the training set. Therefore, we sample 50,000 random projects with at least three files from the StarCoder dataset and count the frequency of calls to functions defined within the project by traversing the abstract syntax tree. In Fig. A.1, we compare the number of calls within individual files to the number of calls within the project as a whole. The number of calls increases with the size of the project or files, and the number of calls within the project is greater than the number of calls within individual files. Consequently, we expect that cross-file code retrieval can improve the quality of code completion, especially for larger projects.

Appendix B. RepoEval modification details

The `line` and `api` datasets from RepoEval (Zhang et al., 2023b) each contain 1600 examples. Every example in `line` is composed of a `prompt` input string and a `ground_truth` output string. Our modified dataset `lineR` is constructed from `line` by expanding each original example's `prompt` with a random number of characters from `ground_truth` and shrinking `ground_truth` accordingly. We are careful that at least one non-whitespace character from `ground_truth` is included into the new `prompt` to avoid expanding `prompt` only by the leading indentation whitespace. The construction of `apiR` from `api` is similar to that of `lineR` from `line`.

Appendix C. Chunk embeddings with BERT

We test the impact of using the BERT encoder versus CodeT5+ on a smaller dataset (6.7 % of the total training set) and find that using the larger BERT encoder for chunk embeddings does not result in significant changes in test metrics – see Table C.1. The comparison is not exhaustive due to the smaller training set and shorter training duration.

Table C.1
Comparison of model training on a smaller training dataset using BERT and CodeT5+ for computing chunk embeddings. GPT-2 tokenization is used.

Model	Encoder	PPL	R_1	R_5	MRR_5
GPT_6^{GPT}	/	2.96	77.9	89.4	82.6
$RETRO_6^{GPT}$	BERT	2.95	78.0	89.5	82.7
$RETRO_6^{GPT}$	CodeT5+	2.95	78.0	89.4	82.7

Appendix D. Indexing chunk embeddings

Due to the large number of chunks in the database, exhaustive searching for the exact k-nearest chunks is not feasible, so we resort to *approximate* searching instead. As a consequence of approximate search, we seek a compromise between the search accuracy, search speed, index building speed, and index memory usage. In our implementation, we use the Faiss library (Johnson et al., 2021) to index the chunk embeddings.[4] The search is sped up using IVF indexing (Johnson et al., 2021) in combination with the HNSW (Malkov & Yashunin, 2020) algorithm. Additionally, the vectors are compressed with the PQ (Jegou, Douze, & Schmid, 2008) vector codec and the OPQ (Ge, He, Ke, & Sun, 2013) transformation. Table D.1 summarizes the space requirements for storing the database and retrieval index based on the tokenization method used.

Table D.1
Comparison of the retrieval database sizes using two different vocabularies for tokenization. The total size of all files before tokenization is 62 GiB.

Tokenization	# Chunks	# Tokens	Tokenized file size	Index size
GPT-2	4.5×10^8	27×10^9	53 GiB	18 GiB
StarCoder	2.8×10^8	18×10^9	33 GiB	12 GiB

Indexing effectiveness. To evaluate the effectiveness of approximate nearest chunk retrieval using the index constructed with Faiss, we compare the L_2 distance between the embedding of a chunk and the embedding of the nearest chunk from the retrieval database returned by the index. For comparison, we observe the distributions of L_2 distances between the embeddings of retrieved chunks and chunks from the database, training set, test set, and chunks composed of random tokens. All chunks contain 64 tokens, and we sample 10 0000 chunks from each set. The distribution results are shown in Fig. D.1. The right side of the figure shows approximate distances as returned by Faiss, which are not exact due to the compression and transformation of embeddings by the indexing. As expected, retrieving chunks from the database itself is nearly error-free (98 % accuracy), while searching with random chunks returns the largest distances. Retrieving chunks from the training set results in smaller distances compared to retrieving chunks from the test set, as the database is constructed by partitioning the training set into consecutive non-overlapping chunks.

[4] In Faiss, the used index can be compactly described by the string `OPQ32_128,IVF1048576_HNSW32,PQ32`.

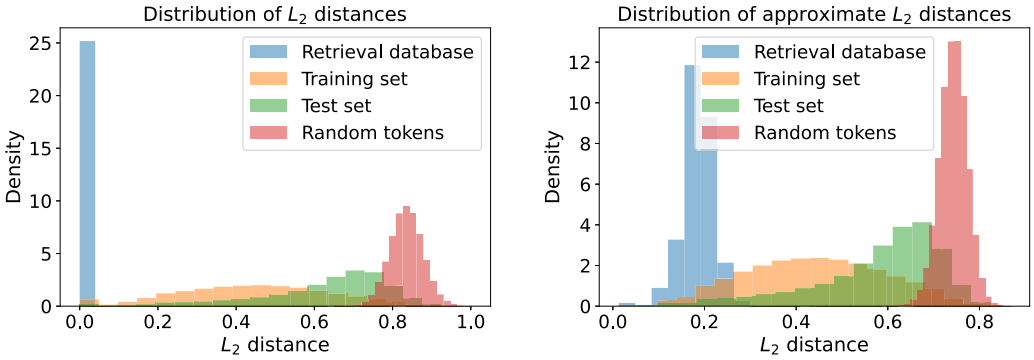


Fig. D.1. Distribution of L_2 distances between chunk embeddings and their retrieved neighbors. The left figure shows exact distances, while the right figure shows approximate distances as returned by the Faiss index.

References

Athiwaratkun, B., Wang, S., Shang, M., Tian, Y., Wang, Z., Gonugondla, S., Gouda, S. K., Kwiatkowski, R., Nallapati, R., & Xiang, B. (2024). Token alignment via character matching for subword completion. In *ACL Findings 2024*.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., & Sutton, C. (2021). Program synthesis with large language models. arXiv:2108.07732

Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., Van Den Driessche, G.B., Lespiau, J.B., Damoc, B., Clark, A., De Las Casas, D., Guy, A., Menick, J., Ring, R., Hennigan, T., Huang, S., Maggiore, L., Jones, C., Cassirer, A. … Sifre, L. (2022). Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th international conference on machine learning* (pp. 2206–2240). (*vol. 162*). Proceedings of Machine Learning Research.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C. … Amodei, D. (2020). Language models are few-shot learners. In *Advances in neural information processing systems* (pp. 1877–1901). (vol. *33*).

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S. … Zaremba, W. (2021). Evaluating large language models trained on code. arXiv:2107.03374

Dagan, G., Synnaeve, G., & Roziere, B. (2024). Getting the most out of your tokenizer for pre-training and domain adaptation. In *Forty-first international conference on machine learning*.

Devlin, J., Chang, M.W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)* (pp. 4171–4186). https://doi.org/10.18653/v1/N19-1423

Ding, Y., Wang, Z., Ahmad, W., Ding, H., Tan, M., Jain, N., Ramanathan, M.K., Nallapati, R., Bhatia, P., Roth, D., & Xiang, B. (2023). CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion. In *Advances in neural information processing systems* (pp. 46701–46723). (vol. *36*).

Donahue, C., Lee, M., & Liang, P. (2020). Enabling language models to fill in the blanks. In *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 2492–2501). https://doi.org/10.18653/v1/2020.acl-main.225

Efron, B. (1987). Better bootstrap confidence intervals. *Journal of the American Statistical Association*, *82*(397), 171–185. https://doi.org/10.1080/01621459.1987.10478410

Evtikhiev, M., Bogomolov, E., Sokolov, Y., & Bryksin, T. (2023). Out of the BLEU: How should we assess quality of the Code Generation models? *Journal of Systems and Software*, *203*, 111741. https://doi.org/10.1016/j.jss.2023.111741

Ge, T., He, K., Ke, Q., & Sun, J. (2013). Optimized product quantization for approximate nearest neighbor search. In *Proceedings of the 2013 IEEE conference on computer vision and pattern recognition* CVPR '13 (p. 2946–2953). https://doi.org/10.1109/CVPR.2013.379

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., LIU, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S.K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., & Zhou, M. (2021). GraphCodeBERT: Pre-training Code Representations with Data Flow. In *International conference on learning representations*.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y.K., Luo, F., Xiong, Y., & Liang, W. (2024). DeepSeek-coder: When the large language model meets programming–The rise of code intelligence. arXiv:2401.14196

Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. (2020). Retrieval augmented language model pre-training. In *Proceedings of the 37th international conference on machine learning* (pp. 3929–3938).

Harman, D.K. (1995). Overview of the third text retrieval conference (TREC-3). 500. DIANE Publishing.

Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. In *EACL 2021 - 16th Conference of the European chapter of the association for computational linguistics* (pp. 874–880). https://doi.org/10.18653/v1/2021.eacl-main.74

Jegou, H., Douze, M., & Schmid, C. (2008). Hamming embedding and weak geometric consistency for large scale image search. In *Computer vision – ECCV* (pp. 304–317). https://doi.org/10.1007/978-3-540-88682-2_24

Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, *7*(3), 535–547. https://doi.org/10.1109/TBDATA.2019.2921572

Jurafsky, D., & Martin, J.H. (2009). Speech and language processing (2nd Edition). Upper Saddle River, NJ, USA: Prentice-Hall, Inc.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. arXiv:2001.08361

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)* (pp. 6769–6781). https://doi.org/10.18653/v1/2020.emnlp-main.550

Kasneci, E., Sessler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günnemann, S., Hüllermeier, E., Krusche, S., Kutyniok, G., Michaeli, T., Nerdel, C., Pfeffer, J., Poquet, O., Sailer, M., Schmidt, A., Seidel, T. … Kasneci, G. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, *103*, 102274. https://doi.org/10.1016/j.lindif.2023.102274

Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., & Lewis, M. (2020). Generalization through memorization: Nearest neighbor language models. In *International conference on learning representations*.

Kingma, D.P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd International conference on learning representations*.

Levenshtein, V.I. et al. (1966). Binary codes capable of correcting deletions, insertions, and reversals. In *Soviet physics doklady* (pp. 707–710). Soviet Union (vol. *10*).

Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast inference from transformers via speculative decoding. In *Proceedings of the 40th international conference on machine learning* (pp. 19274–19286).

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in neural information processing systems* (pp. 9459–9474). (vol. *33*).

Li, R., allal, L.B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Jia, L.I., Chim, J., Liu, Q., Zheltonozhskii, E., Zhuo, T.Y., Wang, T., Dehaene, O., Lamy-Poirier, J., Monteiro, J., Gontier, N., Yee, M.H. … de Vries, H. (2023). StarCoder: May the source be with you!. *Transactions on Machine Learning Research*.

Liang, J.T., Yang, C., & Myers, B.A. (2024). A large-scale survey on the usability of AI programming assistants: Successes and challenges. In *Proceedings of the IEEE/ACM 46th international conference on software engineering* ICSE '24. https://doi.org/10.1145/3597503.3608128

Liu, T., Xu, C., & McAuley, J. (2024). RepoBench: Benchmarking repository-level code auto-completion systems. In *The twelfth international conference on learning representations*.

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., & Zhu, C. (2023). G-Eval: NLG Evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 2511–2522). https://doi.org/10.18653/v1/2023.emnlp-main.153

Lu, S., Duan, N., Han, H., Guo, D., Hwang, S.w., & Svyatkovskiy, A. (2022). ReACC: A retrieval-augmented code completion framework. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 6227–6240). https://doi.org/10.18653/v1/2022.acl-long.431

Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., Li, G., Zhou, L., Shou, L., Zhou, L., Tufano, M., Gong, M., Zhou, M., Duan, N., Sundaresan, N. … Liu, S. (2021). CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Thirty-fifth conference on neural information processing systems datasets and benchmarks track*.

Malkov, Y.A., & Yashunin, D.A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *42*(4), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., Phanishayee, A., & Zaharia, M. (2021). Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. https://doi.org/10.1145/3458817.3476209

Parvez, M.R., Ahmad, W., Chakraborty, S., Ray, B., & Chang, K.W. (2021). Retrieval augmented code generation and summarization. In *Findings of the association for computational linguistics: EMNLP* (pp. 2719–2734). https://doi.org/10.18653/v1/2021.findings-emnlp.232

Patwardhan, N., Marrone, S., & Sansone, C. (2023). Transformers in the real world: A survey on NLP applications. *Information, 14*(4). https://doi.org/10.3390/info14040242

Press, O., & Wolf, L. (2017). Using the output embedding to improve language models. In *Proceedings of the 15th conference of the European chapter of the association for computational linguistics: Volume 2, short papers* (pp. 157–163).

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Blog, 1*(8), 9.

Ram, O., Levine, Y., Dalmedigos, I., Muhlgay, D., Shashua, A., Leyton-Brown, K., & Shoham, Y. (2023). In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics, 11*, 1316–1331. https://doi.org/10.1162/tacl_a_00605

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferrer, C.C., Grattafiori, A., Xiong, W., Défossez, A., … Synnaeve, G. (2024). Code Llama: Open foundation models for code. arXiv:2308.12950

Semenkin, A., Sokolov, Y., & Vu, E. (2024). Context composing for full line code completion. In *The IDE workshop at the 46th international conference on software engineering*.

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 1715–1725). https://doi.org/10.18653/v1/P16-1162

Shapkin, A., Litvinov, D., Zharov, Y., Bogomolov, E., Galimzyanov, T., & Bryksin, T. (2024). Dynamic retrieval-augmented generation. arXiv:2312.08976

Shazeer, N. (2020). GLU variants improve transformer. arXiv:2002.05202

Shrivastava, D., Larochelle, H., & Tarlow, D. (2023). Repository-level prompt generation for large language models of code. In *Proceedings of the 40th international conference on machine learning* (pp. 31693–31715).

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing, 568*, 127063. https://doi.org/10.1016/j.neucom.2023.127063

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). LLaMA: Open and efficient foundation language models. arXiv:2302.13971

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Proceedings of the 31st international conference on neural information processing systems* NIPS'17 (p. 6000–6010).

Wang, B., Ping, W., Xu, P., McAfee, L., Liu, Z., Shoeybi, M., Dong, Y., Kuchaiev, O., Li, B., Xiao, C., Anandkumar, A., & Catanzaro, B. (2023a). Shall we pretrain autoregressive language models with retrieval? A comprehensive study. In *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 7763–7786). https://doi.org/10.18653/v1/2023.emnlp-main.482

Wang, Y., Le, H., Gotmare, A., Bui, N., Li, J., & Hoi, S. (2023b). CodeT5+: Open code large language models for code understanding and generation. In *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 1069–1088). https://doi.org/10.18653/v1/2023.emnlp-main.68

Wu, D., Ahmad, W., Zhang, D., Ramanathan, M.K., & Ma, X. (2024). REPOFORMER: Selective retrieval for repository-level code completion. In *ICML 2024*.

Zhang, A., Lipton, Z.C., Li, M., & Smola, A.J. (2023a). Dive into deep learning. Cambridge University Press.

Zhang, F., Chen, B., Zhang, Y., Keung, J., Liu, J., Zan, D., Mao, Y., Lou, J.G., & Chen, W. (2023b). RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 2471–2484). https://doi.org/10.18653/v1/2023.emnlp-main.151